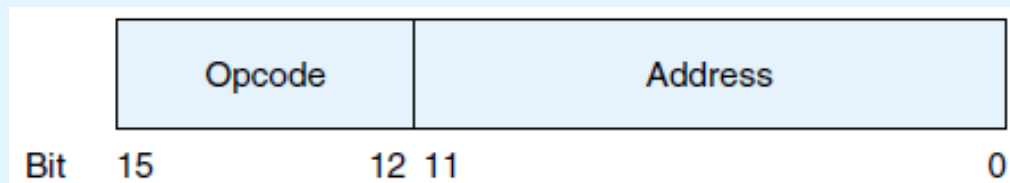


“The Essentials of Computer Organization and Architecture”

Unit 3 – Exercises

MD1 Decipher the following MARIE machine language instructions (write the assembly language equivalent):

- i) 0010000000000111
- ii) 10010000000001011
- iii) 00110000000001001



Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT1 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

Address	Label	Instruction
100		Load A
101		Add One
102		Jump S1
103	S2,	Add One
104		Store A
105		Halt
106	S1,	Add A
107		Jump S2
108	A,	HEX 0023
109	One,	HEX 0001

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT2 Solve again exercise MAT1, now assuming the program begins at memory address 200_{16} .

MAT3 a) Assemble the following program, and
b) provide a trace for its first 4 instructions:

ORG 100

```

If,      100 Load      X      /Load the first value
          101 Subt       Y      /Subtract the value of Y, store result in AC
          102 Skipcond   400    /If AC=0 (X=Y), skip the next instruction
                                   /Note: This is Skipcond 01 in the book
Then,    103 Jump       Else    /Jump to Else part if AC is not equal to 0
          104 Load      X      /Reload X so it can be doubled
          105 Add        X      /Double X
          106 Store      X      /Store the new v
          107 Jump       Endif   /Skip over the f
Else,    108 Load      Y      /Start the else
          109 Subt       X      /Subtract X from
          10A Store      Y      /Store Y-X in Y
Endif,   10B Halt        /Terminate progr
X,       10C Dec         12     /Assume these va
Y,       10D Dec         20
  
```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

MAT4 Solve again exercise MAT3, now assuming the program begins at memory address 400_{16} .

MAT5 a) Assemble the following program, and
b) provide a trace for its first 2 instructions:

```

                                ORG 100
100  Load      X      /Load the first number t
101  Store      Temp   /Use Temp as a parameter
102  JnS        Subr   /Store return address, j
103  Store      X      /Store first number, dou
104  Load      Y      /Load the second number
105  Store      Temp   /Use Temp as a parameter
106  JnS        Subr   /Store return address, j
107  Store      Y      /Store second number, dc
108  Halt                               /End program
X,   109  Dec      20
Y,   10A  Dec      48
Temp, 10B  Dec      0
Subr, 10C  Hex      0      /Store return address he
10D  Clear                               /Clear AC as it was mod
10E  Load      Temp   /Actual subroutine to do
10F  Add         Temp   /AC now hold double the
110  JumpI      Subr   /Return to calling code
                                END
```

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

Instruction Number (hex)	Instruction
0	JnS X
A	Clear
B	AddI X
C	JumpI X
D	LoadI X
E	StoreI X

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300_{16} .

1. Assemble the program under these new conditions and present:
 - a) The symbol table resulting from the 1st assembly stage (fill the table by the order in which the symbols are found in the code during the assembly process);
 - b) The listing of opcodes (in hexadecimal) and symbols that result from the 1st assembly stage;
 - c) The listing with the final result of the assembly process, in hexadecimal.

MAT6 Consider again the program of exercise MAT5, but assume the program begins at address 300_{16} .

2. Based on the answer to question 1, trace the first four instructions executed by the program and present, in the table bellow, each instruction (by the execution order) and the final value of the the registers MAR, MBR, PC, IR and AC, right after the execution of that instruction.

	Instruction 1:	Instruction 2:	Instruction 3:	Instruction 4:
MAR				
MBR				
PC				
IR				
AC				

MC1 Write the following code segment in MARIE assembly language:

```
int X=20, Y=10;  
if (X > 1) {  
    Y = X + X;  
    X = 0;  
}  
Y = Y + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Write the following code segment in MARIE assembly language:

```
int X = 1;  
while (X < 10)  
    X = X + 1;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC2 Write the following code segment in MARIE assembly language:

Note:

```
int X = 1;  
while (X < 10)  
    X = X + 1;
```

<=>

```
int X = 1;  
Loop: if (X < 10) {  
    X = X + 1;  
    goto Loop;  
}
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Write the following code segment in MARIE assembly language:

```
int X = 1;  
do {  
    X = X + 1;  
} while (X < 10)
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC3 Write the following code segment in MARIE assembly language:

Note:

```
int X = 1;  
do {  
    X = X + 1;  
} while (X < 10)
```

<=>

```
int X = 1;  
Loop: X = X + 1;  
    if (X < 10)  
        goto Loop;
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC4 Write the following code segment in MARIE assembly language:

```
int X = 1, Sum = 0;
while (X < 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC5 Solve again exercise MC4, but this time with $X \leq 11$, instead of $X < 11$.

```
int X = 1, Sum = 0;
while (X <= 11) {
    Sum = Sum + X;
    X = X + 1;
}
printf("%d", Sum);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

MC6 Write the following code segment in MARIE assembly language:

```
int i, a[5]={'0','2','4','6','8'};

for(i=0; i<5; i++)
    printf("%c", a[i]);
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC6 Write the following code segment in MARIE assembly language:

Note:

```
int i, a[5]={'0','2','4','6','8'};
for(i=0; i<5; i++)
    printf("%c", a[i]);
```

<=>

```
int i=0;
int a[5]={'0','2','4','6','8'};
Loop: if (i < 5) {
        printf("%c", a[i]);
        i = i + 1;
        goto Loop;
    }
```

Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Instruction Number (hex)	Instruction	Meaning
0	JnS X	Store the PC at address X and jump to X + 1.
A	Clear	Put all zeros in AC.
B	AddI X	Add indirect: Go to address X. Use the value at X as the actual address of the data operand to add to AC.
C	JumpI X	Jump indirect: Go to address X. Use the value at X as the actual address of the location to jump to.
D	LoadI X	Load indirect: Go to address X. Use the value at X as the actual address of the operand to load into the AC.
E	StoreI X	Store indirect: Go to address X. Use the value at X as the destination address for storing the value in AC.

MC7 What are the potential problems with the following assembly code fragment (implementing a subroutine) written to run on MARIE? The subroutine assumes its parameter is in AC and doubles its value. The Main part of the program includes a sample call to the subroutine. You can assume this fragment is part of a larger program.

```

Main,      Load X
           Jump Sub1

Sret,      Store X
...
Sub1,      Add X
           Jump  Sret
```

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Subroutine calling (in C): execution sequence

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);
    E = subr(A, B);
    printf("%d", E);
    return 0;
}
```

x and **y** are the **formal** parameters of the function **subr** (i.e., they are place-holders in the signature / declaration of the function).

```
int subr ( int x, int y )
{
    int res;

    res = x - y;
    return res;
}
```

A and **B** are the **actual** parameters of the function **subr** (i.e., they are the values that are actually passed to the function)

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E=A-B$ and $F=C-D$.

Subroutine calling (in C): parameter passing

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E = subr(A, B);

    printf("%d", E);
    return 0;
}
```

```
int subr(int x, int y)
{
    int res;

    res = x - y;
    return res;
}
```

The value of **A** is copied to the variable **x** of function **subr**.

The value of **B** is copied to the variable **y** of function **subr**.

MC8 Write a MARIE subroutine to subtract two numbers and use the subroutine to help calculate $E = A - B$ and $F = C - D$.

Subroutine calling (in C): result returning

```
int main()
{
    int A, B, E;
    scanf("%d%d", &A, &B);

    E ← subr(A, B);

    printf("%d", E);
    return 0;
}
```

```
int subr(int x, int y)
{
    int res;

    res = x - y;
    return res;
}
```

The value of **res** is copied to the variable **E** of function **main**.

MC9 Write a MARIE program using a loop that multiplies two positive numbers by using repeated addition. For example, to multiple 3 x 6, the program would add 3 six times, or $3+3+3+3+3+3$.

Hint: start by writing the program in C and then translate this program to MARIE assembly.

MC10 Write a program in MARIE assembly that uses a subroutine to evaluate $G = (E = A \times B) + (F = C \times D)$.

Hint: reuse the assembly code from exercise MC9 to implement the subroutine.

MC11 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) A subroutine “Min” to get the minimum of two integers
- b) A subroutine “Max” to get the maximum of two integers
- c) A subroutine “Sym” to get the symmetric of one integer
- d) A subroutine “Div” to get the integer quotient of the division of two integers
- e) A subroutine “Mod” to get the integer remainder of the division of two integers
- f) A subroutine “Prime” to check if an integer is prime

Note: to prevent name clashing (collision), each variable is prefixed with the name of its scope (e.g., Min_X is the X variable of function Min).

MC12 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) **A subroutine “MinArray” to get the minimum of an array of integers**
- b) **A subroutine “MaxArray” to get the maximum of an array of integers**
- c) **A subroutine “ShowSymArray” that shows the symmetric of each element of an array**
- d) **A subroutine “TestPrimeArray” that scans an array of integers and, for each integer, shows 1 (0), if the integer is (not) prime**

MC13 Write the following subroutines in MARIE assembly (and small programs that use them):

- a) **A subroutine “MakeSymArray” that produces an array of integers that are symmetric to the integers of another array**
- b) **A subroutine “MakelsPrimeArray” that produces an array whose cells will be 1 (= True) or 0 (= False) if the corresponding cells of another array of integers contain a prime number, or not**

MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

1. Consider the following program in MARIE assembly:

```
                ORG 100
Loop,           Load Counter
                Subt Num
                Skipcond 800
                Jump LoopBody
                Jump LoopEnd
LoopBody,       Load Counter
                Add One
                Store Counter
                Jump Loop
LoopEnd,        Halt
Counter,        Dec 2
Num,            Dec 10
One,            Dec 1
```

Opcode	Instruction
1	<u>Load</u> X
2	<u>Store</u> X
3	<u>Add</u> X
4	<u>Subt</u> X
5	Input
6	Output
7	<u>Halt</u>
8	<u>Skipcond</u> X X=000: <u>AC</u> <0? X=400: <u>AC</u> ==0? X=800: <u>AC</u> >0?

**MARIE
Instruction Set**

Opcode	Instruction
9	<u>Jump</u> X
0	<u>Jns</u> X
A	<u>Clear</u>
B	<u>AddI</u> X
C	<u>JumpI</u> X
D	<u>LoadI</u> X
E	<u>StoreI</u> X

1.1 Assemble the program and fill in the following tables
(note: all tables have the exact number of lines needed)

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1.1.a The Symbol Table that results from the 1st stage of assemblage (fill the table by the order in which the symbols are found in the source code during the assemblage algorithm; show the addresses in hexadecimal):

Symbol Table

Symbol	Address

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

1.1.b List of the addresses of the memory positions used, and its content (final result of the assemblage), in hexadecimal:

Address	Content

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

2. Based on the answer to question 1.1.b), trace the first five instructions effectively executed by the program. Then, in the table bellow, present each instruction (Inst1 to Inst5, by the order of execution) and the final value (in hexadecimal) of the registers MAR, MBR, PC, IR e AC, right after the execution of that instruction:

Instruction / CPU Register	Inst1:	Inst2:	Inst3:	Inst4:	Inst5:
MAR:					
MBR:					
PC:					
IR:					
AC:					

MATC1 – Consolidated Exercise

(Assemblage, Tracing and Coding)

3. The numbers of Lucas are generated by the formula $L_n = L_{n-2} + L_{n-1}$, with $L_0 = 2$ and $L_1 = 1$, and each of the next numbers being equal to the sum of the two previous Lucas numbers. For instance, the first 5 Lucas numbers are: 2, 1, 3 (=1+2), 4 (=3+1), 7 (=4+3). The C program bellow has a **Lucas** function that returns the N'th Lucas number and a **main** function that makes use of it (note: assume $N \geq 0$ always).

```
int main()
{
    int N, L;

    scanf("%d", &N);

    if (N < 2)
        L = 2 - N;
    else
        L = Lucas(N);

    printf("%d", L);
}
```

```
int Lucas(int Num)
{
    int Number0, Number1, Counter, Current;
    Number0 = 2; Number1 = 1;

    for (Counter = 2; Counter <= Num; Counter++) {
        Current = Number1 + Number0;
        Number0 = Number1;
        Number1 = Current;
    }

    return Current;
}
```


MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.a) Present the MARIE assembly code of function **main**.

```
int main()  
{  
    int N, L;  
  
    scanf("%d", &N);  
  
    if (N < 2)  
        L = 2 - N;  
    else  
        L = Lucas(N);  
  
    printf("%d", L);  
}  
  
int Lucas(int Num)  
{  
    int Number0, Number1, Counter, Current;  
    Number0 = 2; Number1 = 1;  
  
    for (Counter = 2; Counter <= Num; Counter++) {  
        Current = Number1 + Number0;  
        Number0 = Number1;  
        Number1 = Current;  
    }  
  
    return Current;  
}
```

MATC1 – Consolidated Exercise (Assemblage, Tracing and Coding)

3.b) Present the MARIE assembly code of function **Lucas**.

```
int main()  
{  
    int N, L;  
  
    scanf("%d", &N);  
  
    if (N < 2)  
        L = 2 - N;  
    else  
        L = Lucas(N);  
  
    printf("%d", L);  
}
```

```
int Lucas(int Num)  
{  
    int Number0, Number1, Counter, Current;  
    Number0 = 2; Number1 = 1;  
  
    for (Counter = 2; Counter <= Num; Counter++) {  
        Current = Number1 + Number0;  
        Number0 = Number1;  
        Number1 = Current;  
    }  
  
    return Current;  
}
```